Semantic LLM-Directed Fuzzing of DuckDB: Four Methods, One Judge, and 16 Upstream Bugs

PRABHNOOR, IIT Guwahati, India
NIKET GARG, IIT Guwahati, India
JANHVI MITTAL, IIT Guwahati, India

A query optimizer that rewrites a query wrongly returns a wrong answer without any error, so users have no reason to doubt it. Random SQL fuzzers rarely reach the narrow conditions under which a single rewrite rule fires, and a bug found in a released version is often already fixed in the development branch. We present SLDF, a framework of four fuzzing methods for the DuckDB database that run side by side and share one execution judge, one oracle layer and one triage pipeline: (A) classic generic fuzzing as a baseline, (B) fuzzers aimed at features new in DuckDB 2.0, (C) test generators written by a large language model (LLM) and guided by new-code hints and fix dossiers, and (D) a rule-targeted loop. Method D mines a *rule card* for each optimizer rule from its source code and fix history, asks the LLM for a bug hypothesis and a generator program, and accepts the generator only if executing it shows that the target rule fired. With these methods we filed 16 issues on DuckDB 2.0.0: 8 silent wrong results, 5 internal errors and 3 spurious errors. Maintainers have fixed 4 of them, 9 more have open fix pull requests (one of them ours), and Method D alone found 12. One unattended run of Method D spent 40 LLM calls on 13 targets and produced 9 candidates that became 6 issues. Generic fuzzing with 70,980 SQLsmith queries found no new bug.

CCS Concepts: • **Software and its engineering → Software testing and debugging**.

Additional Key Words and Phrases: fuzzing, database testing, query optimizer, differential testing, large language models, DuckDB

ACM Reference Format:
Prabhnoor, Niket Garg, and Janhvi Mittal. 2026. Semantic LLM-Directed Fuzzing of DuckDB: Four Methods, One Judge, and 16 Upstream Bugs. In *Proceedings of ACM International Conference on the Foundations of Software Engineering (Format Template) (FSE (CS-5187 Term Project)).* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Suppose a data analyst filters a table of measurements with `WHERE -d > 'nan'::DOUBLE`. The database plans the query, its optimizer rewrites the predicate into a cheaper form, and the analyst receives rows. Nothing fails and no warning appears. If the rewrite was wrong, the analyst has no way to notice. Database testing tools such as SQLancer [Rigger and Su 2020a,b] and grammar-based fuzzers such as SQLsmith [Seltenreich 2016] and Squirrel [Zhong et al. 2020] were built for this class of *logic bug*. They generate many queries and compare each answer with an answer that should be equal.

These tools work best on code that random queries reach often. An optimizer, however, is a long list of small rewrite rules, and each rule fires only under narrow conditions: a constant of a specific type, a cast next to a comparison, a join key of a list type. A rule that fires on one query in ten thousand is tested rarely, and its edge cases (type limits, signed zero, NaN) more rarely still. A second problem is timing. Projects such as DuckDB [Raasveldt and Mühleisen 2019] are fuzzed continuously by their developers and by other researchers, so a bug found in a release is often

Authors' Contact Information: Prabhnoor, IIT Guwahati, Guwahati, India; Niket Garg, IIT Guwahati, Guwahati, India; Janhvi Mittal, IIT Guwahati, Guwahati, India.

FSE (CS-5187 Term Project), IIT Guwahati
2026. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

```
CREATE TABLE f(d DOUBLE);
INSERT INTO f VALUES (1.5), (-2.5), ('nan'), ('inf');
SELECT d FROM f WHERE -d > 'nan'::DOUBLE ORDER BY d;
  -- DuckDB 2.0.0:  -2.5, 1.5, inf, nan     (4 rows, wrong)
  -- expected:      no rows                  (NaN is the largest value)
SET disabled_optimizers = 'expression_rewriter';
SELECT d FROM f WHERE -d > 'nan'::DOUBLE ORDER BY d;   -- no rows (correct)
```

Fig. 1. Issue #28052, found by an LLM-written generator. One rewrite rule turns an always-false predicate into an always-true one. No error is raised; only a second run with the rule switched off shows the difference.

already fixed on the development branch. In our first month, every real bug our tools found in DuckDB 1.5.6 had already been reported or fixed upstream (Section 5.2).

We focus on a third problem that starts where these tools stop: *directing* the search at one optimizer rule at a time, and proving that the generated tests actually exercise that rule. Large language models (LLMs) can read source code and write programs, which makes directed generation possible [Xia et al. 2024; Yang et al. 2024]. But an LLM also reports success it did not achieve. In an early run of our pipeline, 10 of 11 generators that an LLM agent wrote and described as thorough produced exactly 8 queries per case and 5 to 12 distinct plans (Section 5.3).

A real example. Figure 1 shows issue #28052, which one of our LLM-written generators found in DuckDB 2.0.0. DuckDB orders NaN above every other value, so `-d > NaN` is false for every row. The rule `MoveUnaryMinusRule` rewrites `-x > c` into `x < -c`. Its guard handles a NaN *column value* but not a NaN *constant*, so the query returns all 4 rows instead of none. With the expression rewriter switched off, DuckDB returns the correct empty result, and DuckDB 1.5.6 was also correct. The maintainers merged a fix one day after we filed (PR #28071). This is not a single case: 8 of the 16 issues we filed are silent wrong results (Table 1).

Closest to our work, WhiteFox [Yang et al. 2024] lets an LLM read compiler optimization code and write tests for it, and directed greybox fuzzing [Böhme et al. 2017] steers inputs toward chosen code locations. However, WhiteFox targets compilers and trusts the LLM's description of what triggers an optimization, and directed greybox fuzzers need distances over a control-flow graph, which is costly to compute for a large C++ database. Neither checks with the database itself that a generated SQL test fired the intended rewrite.

Our approach. We present SLDF (Semantic LLM-Directed Fuzzing), a framework of four methods that run side by side on the same infrastructure (Section 3):

(1) **Method A, classic fuzzing**, as a baseline and safety net: SQLsmith on a sanitizer build, differential sweeps of DuckDB's own tests, and replay of past bug reports (Section 3.2).
(2) **Method B, feature-targeted fuzzers** for features that are new in DuckDB 2.0, such as triggers (Section 3.3).
(3) **Method C, new-code hints and LLM-written generators**: the LLM receives hints about code that is new or never executed, feature cards and fix dossiers, and writes a generator program (Section 3.4).
(4) **Method D, the rule-targeted loop**: a *rule card* per optimizer rule, mined from its source and fix history, a hypothesis-first LLM generator, and an *engine judge* that accepts the generator only if the rule fires on its queries (Section 3.5).

All four methods feed one oracle layer and one triage pipeline. A person reads every surviving candidate before anything is filed.

Results. Over five weeks we filed 16 issues on DuckDB 2.0.0. Maintainers fixed 4 (#27759, #27850, #28052, #28562), 9 more have open fix pull requests, and one of those (#28547) is ours. Method D found 12 of the 16. One unattended run of Method D spent 40 LLM calls on 13 targets, ran 32 generator rounds and turned 597 raw differences into 9 triaged candidates and 6 issues (Table 4). The main lesson is that the LLM is useful only when the engine checks its work, and that the check must not be the only gate: four generators that found bugs failed our own diversity floor, so we triage the hits of every generator but count only judged generators as test coverage.

Contributions.

- A framework of four complementary fuzzing methods for a database optimizer that share one judge, one oracle layer and one triage pipeline (Section 3).
- Rule cards: per-rule targets mined from source code and fix history, including the early exits under which a rewrite fires and the places where it builds values (Section 3.5).
- An engine judge that measures rule-level reach and rejects generators that only claim it, checked by positive and negative controls (Sections 3.1 and 5.3).
- 16 upstream issues on DuckDB with minimal reproducers, 4 of them fixed, and one fix pull request (Section 5). All code and data are public (Section 9).

2 Background

DuckDB and its optimizer. DuckDB [Raasveldt and Mühleisen 2019] is an embedded analytical database written in C++. A query passes through a parser, a binder, an optimizer and a vectorized executor. In DuckDB 2.0.0 the optimizer applies a fixed sequence of 46 passes (for example filter pushdown, join ordering, removal of unused columns, top-N window elimination). One pass, the *expression rewriter*, applies small *rules* (29 source files in `src/optimizer/rule`) to every expression, for example moving constants across a comparison or simplifying a `date_trunc`. The setting `disabled_optimizers` switches off a named pass; a rule can only be switched off together with its whole pass. We call this a *switch*. DuckDB 2.0.0 was released during our project; we tested that release and checked every candidate on the newest development build (*main*).

Logic bugs and oracles. A *logic bug* makes the database return a wrong result without an error. To detect one, a fuzzer needs an *oracle*: a second computation whose result must be equal. Differential testing compares two systems or two configurations of one system. Ternary logic partitioning (TLP) [Rigger and Su 2020b] splits a query by a predicate into three parts that must add up to the original. NoREC [Rigger and Su 2020a] compares an optimized query with a form the optimizer cannot rewrite. Our main oracle is differential: the same query with a switch on and off. An *internal error* is DuckDB's own assertion that an impossible state was reached; a *spurious error* is an error raised for a valid query that should succeed.

Reach. A test for a rule is useful only if the rule fires on it. We say a query *reaches* a target when its plan changes when the target's switch is turned off (*expression-level reach*), and, more strictly, when a call that only the target rule removes disappears from the plan (*rule-level reach*). The fraction of valid queries that reach the target is the *reach rate*.

Terms used later. A *generator* is a short Python program that returns test cases (a setup script and queries); an LLM writes it, our harness runs it. A *hit* is a raw difference between configurations. A *candidate* is a hit that survived automatic triage. An *issue* is a candidate a person checked and filed upstream.

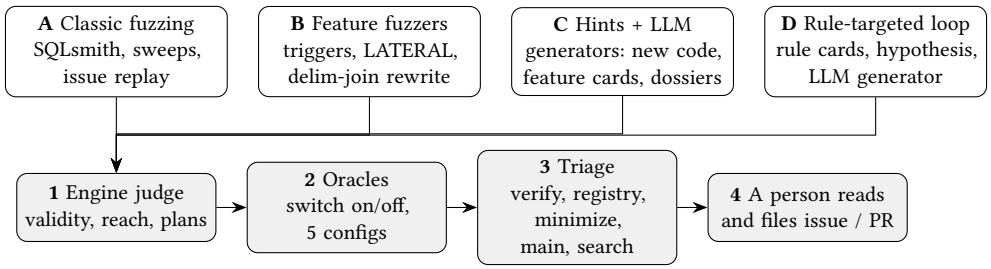


Fig. 2. SLDF: four fuzzing methods run side by side and share one core. Generators written by an LLM (Methods C and D) pass the engine judge (box 1) first, which also returns measured feedback for the next round of Method D. Every hit goes through the same oracles and triage, and a person decides what is filed.

3 Approach

SLDF consists of four fuzzing methods and one shared core (Figure 2). The methods differ in *where* they look and *who* writes the tests. The core decides *whether* a test is meaningful and *whether* a difference is a bug. We ran the methods side by side, not one after the other: each covers a blind spot of the others, and a new method can be added without changing the core. We first describe the core (Section 3.1), then the four methods (Sections 3.2–3.5).

3.1 The Shared Core

① *The engine judge.* An LLM-written generator is run on DuckDB, and its quality is measured by the database, not reported by the LLM. For a generator with target switch s, the judge runs every query once with s on and once with s off and records: the *valid rate* (queries DuckDB accepts); the *reach rate* (Section 2); the number of distinct physical operator trees, read from `EXPLAIN (FORMAT JSON)`; and the number of distinct query *skeletons*, that is, queries with identifiers and literals removed. A generator is rejected as *low diversity* if any hard flag holds: fewer than $\max(30, 0.02n)$ skeletons for n valid queries; fewer than $\max(15, 0.03c)$ operator trees for c cases; a reach rate below 30%; fewer than half of the valid queries taking part in the result comparison; or a rule target without a marker function. For a rule target, reach must be *rule-level*: the rule card names a function call that only this rule removes, and the call must disappear from the plan. A generator that names the rule but never gives it a bare column passes expression-level reach but fails rule-level reach (Section 5.3).

② *Oracles.* Each query runs under five configurations: the default; the target switch off; `delim_join_as_cte=false` (a rewrite new in 2.0); one thread; and `PRAGMA enable_verification`, which makes DuckDB re-run each query in several internal ways. A *config difference* is a result that differs from the default. An *error difference* is an error raised in one configuration only; if only the optimized run fails, the optimizer turned a valid query into a failing one. Results are compared as multisets after normalizing values close to zero, and queries whose order is undefined are excluded. The generator contract requires unique sort keys, because ties under `ORDER BY` or `row_number()` are the first source of false alarms.

③ *Triage.* Every hit, from any method, passes five automatic steps: (1) *verify*: replay the script alone and in a fresh process, and drop it if the difference disappears; (2) *registry*: compare the signature with our registry of known issues, which labels it a rediscovery, a sibling or new; (3) *minimize*: greedy delta reduction over statements and then over rows of `VALUES` lists, keeping the same verdict; (4) *main*: run the minimized script on the newest development build and drop it if it

is fixed there; (5) *search*: query GitHub for issues and pull requests with the same function, rule or error text. Hits where the optimizer only hides an error (the unoptimized run fails, the optimized one succeeds) are labelled low priority, because DuckDB accepts this behaviour.

④ *The person.* Triage writes a list of candidates with minimized reproducers. A team member reads each one, decides which side is right by reading the rule's code, and writes the issue. Nothing is filed automatically. We grouped siblings with one root cause into one issue.

3.2 Method A: Classic Fuzzing

Method A is generic, undirected fuzzing. It runs SQLsmith [Seltenreich 2016] against an AddressSanitizer and UndefinedBehaviorSanitizer build of DuckDB, sweeps the 2,560 SQL scripts we extracted from DuckDB's own test suite under the five configurations, mutates them (boundary values, join swaps, nesting), and replays reproducers mined from closed upstream issues. It needs no LLM and no target. We keep it running as a baseline and as a safety net for memory errors, which the other methods do not look for.

3.3 Method B: Feature-Targeted Fuzzers

Method B aims at features that are new in DuckDB 2.0 and rarely appear in the upstream issue stream. Each fuzzer is hand-written for one feature and has its own oracle. The trigger fuzzer builds random trigger definitions and data-changing statement sequences and checks three things: the final table equals an emulation of the triggers in plain SQL, the result survives a restart, and no internal error occurs. Further fuzzers cover window functions with mixed types, join equivalences (a `NATURAL JOIN` against `INTERSECT ALL`), bottom-up query construction, and the new `delim_join_as_cte` rewrite, which they compare on and off.

3.4 Method C: New-Code Hints and LLM-Written Generators

Method C points an LLM at code that is new or never executed. Its inputs are: (1) *static hints*: 24,816 branches, boundary constants and risky operations extracted from DuckDB's C++ with tree-sitter, of which 80.9% are never executed by the seed corpus of 2,560 test scripts; (2) *new-code hints*: the code that changed between DuckDB 1.5 and 2.0; (3) *feature cards*: a short plain-language description of a new feature and its switches; and (4) *fix dossiers*: the diff and test of a recent fix, so that the LLM can look for a variant the fix missed. The LLM writes a generator under a fixed contract (deterministic output, unique sort keys, a declared target switch), and the engine judge measures it. A one-command, resumable 12-hour run (`targeted_run`) drives the loop and ends with an audit that recomputes every quality verdict from stored counts. We ran Method C with three models: a local 1.5B-parameter model in a pilot, Gemini 2.5 Pro (run by a team member), and Claude in an interactive authoring arm.

3.5 Method D: The Rule-Targeted Loop

Method D directs the search at one optimizer rule at a time. It has four steps.

Step 1: Rule cards. For every rule file and optimizer pass, a script builds a *rule card* from the source and the git history: (a) the commits since March 2026 that touched the file, the fixes among them, and the tests these fixes added (a fix usually covers only the reported case, and the next bug is often in the case next to it); (b) every early exit in the rule's `Apply` or `CanOptimize` function, that is, the exact conditions under which the rewrite fires; (c) every place where the rule builds or converts a value (bounds, folded constants, casts); and (d) its marker function and its switch. Cards are ranked by $score = 2\min(fixes, 6) + \min(commits, 15) + 3\min(sites, 10) + 15[rule] + 5[marker]$,

which favours expression rules that build values and were fixed recently. For DuckDB 2.0.0 the script builds 49 cards.

Step 2: Hypothesis-first generation. The LLM receives one card and reads the real DuckDB source with read-only tools (Read, Grep, Glob). Its working directory is an empty folder, so it never sees our issue drafts, our controls or earlier generators. It must state a bug hypothesis and the preconditions under which the rule fires (`META["hypothesis"]`, `META["preconditions"]`); the judge rejects a rule-targeted generator without them. The prompt gives three recipes that worked for us: take constants from the generated rows (so that = can match) and from a list of 275 literals at the limits of 30 types (type maxima, −0.0, infinity, extreme dates); place the target pattern in varied contexts (`WHERE`, `SELECT`, `CASE`, `FILTER`, `JOIN ON`, subqueries); and keep queries deterministic.

Step 3: Judge and repair. The judge runs the generator on 300 cases. Its measured feedback (valid rate, reach rate, the most common errors, queries that reached the target and queries that did not) becomes the prompt of the next round. Round 1 of a target uses a larger model (Claude Opus), and repair rounds use a smaller one (Claude Sonnet). A target retires after three rounds. A generator that the judge accepts and that produced hits is re-run at scale (1,500 cases).

Step 4: Triage of every hit. All hits go to the shared triage, including hits of generators the judge rejected. We made this choice after we saw that narrow, deep generators find bugs but have few plan shapes (Section 5.3). The judge therefore decides what counts as tested coverage, and triage decides what counts as a bug.

Running unattended. The loop keeps a ledger of targets, rounds and candidates, so a stopped run resumes where it stopped. A failed LLM call is not counted, the loop backs off, and after four failures in a row (for example a usage limit) it stops cleanly without retiring targets. A supervisor restarts it after a crash. Before every unit it checks free memory and pauses below 1.5 GB, and it keeps the laptop awake while it runs.

Why not let the LLM write queries directly? A generator is a program: one LLM call yields hundreds of different queries, and the judge can measure the program instead of trusting a handful of examples. In our runs one call produced 300 to 1,500 test cases.

4 Implementation

SLDF is about 11,000 lines of Python in `src/harness`, plus a C++ driver that runs SQL in-process on the sanitizer build. The main parts are: `gen_runner.py` (the engine judge and the five-configuration oracle), `verify_finding.py` and `gen_triage.py` (triage), `known_issues.json` (the registry), `rule_targets.py` (rule cards), `edge_values.py` (the 275 type-limit literals), `hunt_loop.py` with `hunt_start.sh` (the unattended loop of Method D), `targeted_run.py` (Method C), and the fuzzers of Methods A and B. The LLM in Method D is Claude Code in headless mode (`claude -p`) with only the Read, Grep and Glob tools allowed. `controls/check_controls.py` runs the positive and negative controls after every change to the judge or the triage. All runs pin the time zone to UTC, because DuckDB's default time zone follows the process environment.

5 Evaluation

We answer four research questions.

RQ1 Effectiveness. How many previously unknown bugs does SLDF find in DuckDB 2.0.0, of which kinds, and how do maintainers respond?

RQ2 Methods. What does each of the four methods contribute, and at what cost?

Table 1. All 16 issues we filed on DuckDB. *Method*: the method of Section 3 that found it (D-s: interactive session, D-l: unattended loop). *Kind*: wrong result (silent), internal error (an assertion or INTERNAL Error), or spurious error (a valid query fails). *Status* on 2026-11-13; fix PRs are by maintainers or other contributors, except #28547 (ours).

Issue	Meth.	Kind	What goes wrong	Status
#27759	B	internal	FOR EACH ROW trigger into a table with a BEFORE INSERT trigger	fixed, PR #28698
#27850	C	internal	identical EXISTS blocks under UNION ALL (delim_join_as_cte=false)	fixed, PR #28144
#27904	C	internal	CROSS JOIN LATERAL with a nested LATERAL count(*)	open, PR #27975
#28052	C	wrong	negated DOUBLE compared with a NaN constant	fixed, PR #28071
#28527	D-s	wrong	date_trunc with a sub-day part on a DATE column	open, PR #28528
#28531	D-s	spurious	CAST(timestamp AS DATE) = date at the TIMESTAMP range limits	open, PR #28547
#28532	D-s	internal	TIMESTAMP_MS to TIMESTAMP_S cast at range end invalidates the database	open, PR #28539
#28534	D-s	spurious	date_trunc comparison near the end of the TIMESTAMP range	open
#28535	D-s	internal	UBIGINT constant above BIGINT max in x + c1 = c2	open, PR #28540
#28536	D-l	wrong	regex empty group (?:) at the pattern edge treated as an anchor	open, PR #28538
#28541	D-l	wrong	UHUGEINT constant above HUGEINT max wraps in x + c1 = c2	open, PR #28549
#28542	D-l	wrong	TRY_CAST(date AS TIMESTAMP) comparison returns false instead of NULL	open
#28559	D-l	wrong	top-N window elimination returns the other join side for 0.0 / -0.0	open, PR #28666, #28728
#28562	D-l	wrong	contains(list, '') returns true for lists without an empty string	fixed, PR #28563
#28565	D-l	spurious	CASE guard around a STRUCT field cast ignored	open, reproduced
#28566	D-l	wrong	both sides of a DOUBLE[] join key merged for [0.0] and [-0.0]	open, PR #28728

RQ3 Judge. Does the engine judge separate generators that exercise their target from generators that only claim to?

RQ4 Loop. How efficient is the unattended rule-targeted loop, from LLM calls to filed issues?

Setup. All runs of Methods A, B and D and the Claude arm of Method C used one laptop: an Intel Core i3-1315U (8 hardware threads), 7.4 GB of RAM under WSL2 (Linux 6.18), Python 3.14.4 and the released DuckDB 2.0.0 (source tag `v2.0.0`); Method A on DuckDB 1.5.6 and 2.0.0 used our own sanitizer builds. The LLM of Method D was Claude Code 2.1.323 with Claude Opus for the first round of each target and Claude Sonnet for repair rounds, on a personal subscription. Round 2 of Method C ran on a team member's machine with Gemini 2.5 Pro. Workers ran one at a time with a DuckDB memory limit, because larger settings froze the machine. Every number in this section is produced by `paper/scripts/collect_data.py` and `collect_extra.py` from run logs and from the GitHub state of our issues on 13 November 2026; the outputs are in `paper/data`.

5.1 RQ1: Bugs Found

Table 1 lists the 16 issues we filed between 23 October and 9 November 2026; a team member filed one (#27904) and another team member filed the rest. Each issue has a minimized reproducer, the affected versions, and the result with the responsible switch turned off. By kind, 8 are silent wrong results, 5 are internal errors, and 3 are spurious errors on valid queries. Two of the internal errors are severe: #28532 invalidates the whole database, so every later query in the session fails, and #27759 aborts a trigger cascade.

Maintainers have fixed 4 issues: #28052 (PR #28071), #27850 (PR #28144), #28562 (PR #28563) and #27759 (PR #28698, which also fixes a related wrong result; DuckDB's lead developer closed the issue). For 9 more issues a fix pull request is open; 8 of them were written by maintainers or other contributors after reading our report, and one (#28547 for #28531) is ours. DuckDB's bot labelled 11 of the 16 issues as *reproduced*. None was closed as a duplicate or as expected behaviour.

The bugs cluster in one family: constants at the limits of their type inside optimizer rewrites. The rewrite builds a new value (a bound, a moved constant, a cast) and that value overflows, wraps or loses a NaN or a sign. In #28541, a UHUGEINT constant above the HUGEINT maximum silently

Table 2. What each method ran and what it found. *Work*: tests executed or LLM calls spent. *New*: issues filed that were previously unknown. Method A ran first on DuckDB 1.5.6 and later on 2.0.0; all its real findings were already known upstream.

Method	Input and LLM	Work	New	Fixed
A classic	none	70,980 SQLsmith queries (92.3% valid); 11,235 differential queries; 14,000 sanitizer runs	0	0
B features	hand-written fuzzers for new 2.0 features	trigger, window, join, bottom-up and delim-join fuzzers	1	1
C hints + LLM	static and new-code hints, feature cards, dossiers; 1.5B local model, Gemini 2.5 Pro, Claude	Gemini Round 2: 12 generators, 36,600 cases, 122,590 valid queries; Claude arm	3	2
D rule loop	rule cards; Claude Opus and Sonnet	interactive session; loop runs `smoke1` (2 calls) and `hunt1` (40 calls)	12	1
Total			16	4

wraps when it is moved across =. In #28534, a `date_trunc` bound lies one step past the last valid timestamp. In #28559 and #28566, the signed zero makes `[0.0] = [-0.0]` true although the two values print differently.

Observation 1. SLDF found 16 previously unknown bugs in DuckDB 2.0.0, half of them silent wrong results. Maintainers fixed 4 within three weeks and fix pull requests exist for 9 more; none was rejected.

5.2 RQ2: What Each Method Contributes

Table 2 compares the methods. **Method A** found real failures but no new bug. A differential sweep of 11,235 queries over DuckDB's own tests on DuckDB 1.5.6 surfaced two crashes, both already reported and fixed upstream; a sanitizer hunt over 14,000 runs produced 109 hits with 22 distinct signatures, all of them known bugs or expected behaviour; and a 45-minute SQLsmith campaign of 70,980 queries on the 2.0.0 sanitizer build produced 46 raw anomalies and no new bug. This is the gap that motivated the other methods: generic fuzzing re-finds what upstream's own fuzzers already find.

Method B found our first new bug: the trigger fuzzer found #27759, an internal error when a `FOR EACH ROW` trigger inserts into a table that has a `BEFORE INSERT` trigger. Triggers were new in DuckDB 2.0 and rarely appeared in upstream reports.

Method C found three bugs (#27850, #27904, #28052), two of them now fixed. Its results depend strongly on the model. In the pilot with a local 1.5B-parameter model, 72 of 126 guided queries were valid (57%) and none of 10 targeted code regions was reached. Gemini 2.5 Pro in Round 2 wrote 12 generators that passed the audit and ran 122,590 valid queries without a new bug; its four candidates were two sanitizer-only failures already fixed upstream and two that did not reproduce. The same round shows the cost of directing: the directed generators covered 17.89% of DuckDB's 160,389 instrumented lines, against 38.64% for the baseline corpus, because each generator stays close to one target.

Method D found 12 of the 16 bugs: 5 in an interactive session in which Claude read the optimizer source and its fix history with a person in the loop, and 7 in the unattended loop. Rule cards automate what this session did by hand. These 7 came from 42 LLM calls (Section 5.4).

Table 3. Controls of the judge and the triage. A control must reach its expected verdict after every change to them. *Rule*/*Expr.*: rule-level and expression-level reach rate. *Re-found*: the issue that triage classified the control's hits as a rediscovery of.

Control	Judge verdict	Rule	Expr.	Hits	Re-found
cheat (names rule, never fires it)	rejected	0.00	0.23	0	–
positive (`date_trunc`)	accepted	0.33	0.55	8	#28527
session generator, arithmetic	rejected (diversity)	0.59	0.59	8	#28535
session generator, cast comparison	rejected (diversity)	0.11	0.40	12	#28532
session generator, `date_trunc` edges	rejected (diversity)	0.22	0.56	6	#28534
session generator, timestamp cast	rejected (diversity)	0.35	0.58	2	#28531

Observation 2. The methods are complementary. Generic fuzzing found no new bug, a feature fuzzer found the first one, and directing an LLM at single rules found 12 of 16. A larger model and a narrower target mattered more than more tests: Method C ran more than 120,000 queries in one round without a new bug, and Method D found 6 bugs with 40 calls.

5.3 RQ3: Does the Engine Judge Work?

The judge exists because an LLM reports progress it did not make. In the first unattended Method C run, 10 of 11 generators written by an LLM agent produced exactly 8 queries per case and 5 to 12 distinct plans, and the run's only reported bug was our own #27759 again. Our first judge counted SQL keywords instead of operators, which a red-team review showed could be satisfied without testing anything; we rebuilt it on operator trees from `EXPLAIN (FORMAT JSON)`.

Table 3 shows the controls we run after every change. The negative control names the target rule but never gives it a bare column, so constant folding removes the call before the rule sees it. Its expression-level reach is 0.23, but its rule-level reach is 0.00, and the judge rejects it. The positive control is accepted and its hits are classified as rediscoveries of #28527. The four session generators that found #28531, #28532, #28534 and #28535 still re-find exactly their own bug, so the triage and the registry work end to end.

The same table also shows a limit of the judge. All four session generators fail the diversity floor: they are narrow and deep, with few query skeletons, yet each found a real bug. For this reason Method D triages the hits of every generator. In `hunt1`, the judge accepted 23 of 32 rounds, and all 9 candidates came from accepted rounds; the rejected rounds produced 128 raw hits and no candidate.

Observation 3. Measured rule-level reach rejects a generator that only names its target, and controls that re-find our own bugs show the pipeline still works after every change. The diversity floor is a measure of coverage, not of bug-finding power: narrow generators can find bugs, so their hits must still be triaged.

5.4 RQ4: The Unattended Loop

Table 4 shows the run `hunt1`. It spent 40 LLM calls on 13 targets (12 of them ran at least one round), ran 32 generator rounds with a median valid rate of 95%, and collected 597 raw hits. Triage looked at 138 hits, at most ten per round. It classified 56 as duplicates within the run and 54 as already fixed on the development build. Another 10 were low priority (the optimizer only hid an error), 8 were rediscoveries of known issues and 1 was an artifact. The remaining 9 were new candidates. A person read the 9 candidates: four were the same top-N bug, one was covered by an open upstream pull request, and six became issues. A short earlier run (`smoke1`, 2 calls) gave one more (#28536).

Table 4. The automated loop (Method D, run `hunt1`): most targets give nothing, a few give bugs. *Units*: generator rounds run. *OK*: rounds the engine judge accepted. *Reach*: best fraction of valid queries whose plan changes when the target is switched off. *Hits*: raw differences before triage. *Cand.*: hits that survived triage. *Issues*: filed upstream.

Target	Units	OK	Reach	Hits	Cand.	Issues
`remove_unused_columns`	4	4	1.00	95	2	28565, 28566
`date_trunc_simplification`	2	1	0.61	0	0	–
`move_constants`	2	2	0.98	98	1	28541
`comparison_simplification`	2	1	0.31	189	1	28542
`monotone_preimage`	3	3	0.80	10	0	–
`timestamp_comparison`	3	3	0.77	3	0	–
`string_prefix`	3	2	0.64	1	0	–
`topn_window_elimination`	3	3	0.97	108	4	28559
`in_clause_simplification_rule`	3	1	0.37	60	0	–
`compressed_materialization`	1	0	0.59	0	0	–
`empty_needle_removal`	3	2	0.63	31	1	28562
`list_comprehension_rewrite`	3	1	0.60	2	0	–
Total	32	23		597	9	6

The run was started seven times; after a machine sleep, failed LLM calls and a usage limit, each start resumed from the ledger without repeating a completed round.

Yield is uneven. Five of the 12 targets produced all the candidates, and they share a trait: the rule builds a new value, or it replaces one column with another that compares as equal. Rules whose queries rarely reach them (`in_clause_simplification_rule`, `compressed_materialization`) produced nothing. The triage step saved much work by dropping 54 hits whose bug was already fixed on the development build, for example top-N hits fixed by PR #28405 two days before the run.

Observation 4. The loop turned 40 LLM calls into 6 new issues, about one issue per 7 calls, with a person reading only 9 candidates out of 597 raw hits. Yield comes from a few targets that build values, so rotating targets helps more than longer runs on one target.

6 Discussion

What made directing work. Three choices mattered most. First, targets came from code *and* history: the bugs of Method D sit next to recent fixes (#28541 is the UHUGEINT sibling of the UBIGINT bug #28535, found one day later by the loop on the same rule). Second, the LLM had to state a hypothesis about a value the rule builds, which pushed it toward type limits, NaN and signed zero instead of generic queries. Third, the database judged the result. Without the judge, the LLM's own reports in Method C would have counted trivial generators as thorough.

Why keep generic fuzzing? Method A found no new bug, but it is the only method that looks for memory errors under sanitizers, and its results define the baseline: they show that the bugs of Methods B to D are not ones a generic fuzzer finds in the same time.

Why not report every difference? Most differences are not bugs. Of 138 triaged hits in `hunt1`, only 9 were new candidates. Many of the rest are expected: when the unoptimized plan fails and the optimized plan succeeds, the optimizer only removed work whose error does not matter, which DuckDB accepts. Sending such reports would cost maintainers' time and our credibility. We also grouped bugs with one root cause into one issue and filed at a steady pace.

Limits of the bug family. Most of our bugs belong to one family: constants at type limits in optimizer rewrites. Yield per target falls once a rule's family is exhausted (Table 4). New families need new oracles: TLP and NoREC can find bugs that are present with the switch both on and off, and a cross-version oracle (release against development build) finds regressions directly. Rules that match operators such as `+` or `LIKE` have no marker function, so their reach is measured only at expression level. Measuring the coverage of the target file's lines would give a stronger signal for them. We leave these for future work.

7 Threats to Validity

Internal validity. A difference between two configurations can come from the oracle, not the database: ties under `ORDER BY`, unordered `LIMIT`, `any_value`, row identifiers, floating-point noise near zero, or a time-zone dependence. Each new generator's first hits were usually such oracle errors. We reduce this risk by the generator contract (unique sort keys), result normalization, replay in a fresh process, minimization that keeps the verdict, and a person who decides which side is right by reading the rule's code before filing. Upstream confirmation is the final check: 11 of 16 issues are labelled reproduced and 4 are fixed. The method that found each bug is taken from our notes and run logs; for the issues found before the automated loop existed, the attribution is ours and was not measured. One early bug that we first believed new had been fixed upstream the same day (PR #28405); we caught it before filing because triage checks the newest development build, but such races can recur.

External validity. We tested one database, DuckDB, and mostly one release, 2.0.0, which was new during our project. A new major release probably contains more optimizer bugs than a mature one, so our yield may not transfer to older or more heavily tested systems. Most bugs belong to one family, so the results say little about other bug classes such as concurrency or storage bugs. The LLMs we used change over time, and a later model may behave differently on the same prompts.

Construct validity. We count *filed issues*, not confirmed root causes. An issue that maintainers later close as expected behaviour would reduce the count; none has been so far. Coverage numbers come from one round of Method C and measure lines, which predict bug finding only weakly. Reach is measured by a plan change, which shows that the rewrite fired but not that it fired in the interesting way. Our cost measure counts LLM calls, not tokens or wall time, because the runs were interrupted by usage limits and by the machine sleeping.

8 Related Work

Logic-bug oracles for databases. Pivoted Query Synthesis [Rigger and Su 2020c], TLP [Rigger and Su 2020b] and NoREC [Rigger and Su 2020a] construct queries whose results are known to be related and found many logic bugs in SQLite, MySQL and PostgreSQL. Query Plan Guidance [Ba and Rigger 2023] steers generation toward new query plans, and CERT [Ba and Rigger 2024] finds performance issues through cardinality estimates. These oracles are generic: they apply to any query. We use the simplest differential oracle, switch on against switch off, and spend our effort on *which* queries to run: queries that reach one named rule, written for that rule.

Grammar- and coverage-based SQL fuzzing. RAGS [Slutz 1998] introduced massive stochastic testing of SQL, SQLsmith [Seltenreich 2016] generates random valid queries from a database's schema, and Squirrel [Zhong et al. 2020] mutates an intermediate representation with coverage feedback to keep queries valid. They search broadly and find crashes in frequently executed code. Our Method A uses SQLsmith as a baseline. Unlike these tools, Method D aims at one optimizer rule at a time and verifies that the rule fired.

LLM-based fuzzing. Fuzz4All [Xia et al. 2024] uses an LLM as a general input generator for many languages. WhiteFox [Yang et al. 2024] lets an LLM summarize compiler optimization source code into requirements and then write tests, with feedback from tests that triggered the optimization. CovRL [Eom et al. 2024] rewards LLM mutations by rare coverage. Our Method D is closest to WhiteFox, but differs in three ways: the LLM writes generator programs rather than single tests, it must state a hypothesis about a value the rule builds, and the database, not the LLM, decides whether a rule fired (rule-level reach). We also mine targets from fix history, not only from the current source.

Directed fuzzing. AFLGo [Böhme et al. 2017] directs greybox fuzzing toward target locations by distance in the call and control-flow graphs. We direct at the level of a rewrite rule and use the plan change as a cheap, exact signal that the target was reached, which needs no graph distance and no instrumented build.

9 Conclusion

We presented SLDF, four fuzzing methods for DuckDB that share one engine judge, one oracle layer and one triage pipeline. Generic fuzzing found no new bug; a fuzzer for a new feature found the first one; LLM-written generators guided by hints found three; and the rule-targeted loop, which mines rule cards from source and fix history and lets the database judge an LLM's generators, found twelve. In total we filed 16 issues on DuckDB 2.0.0, 8 of them silent wrong results; 4 are fixed and 9 more have open fix pull requests. The main lesson is that an LLM becomes a useful fuzzer author when its claims are replaced by measurements from the system under test, and when every hit is still triaged by the same pipeline. Next, we plan to add TLP, NoREC and cross-version oracles, coverage of target lines as a second reach signal, and adapters for PostgreSQL and SQLite-compatible engines.

Data Availability

All code, generators, controls, run logs and the scripts that produce every number in this paper are in our repository, https://github.com/PASS-IITG/cs5187-term-project-janhvimittal07. `src/README.md` explains how to build the DuckDB versions we used and how to rerun each method; `paper/scripts/collect_data.py` regenerates `paper/data` and Tables 1 and 4. Each upstream issue in Table 1 contains its minimized reproducer.

Author Contributions

Prabhnoor designed and built the LLM pipeline, the engine judge, the triage, Methods B and D and the unattended loop, ran most experiments, filed 15 of the 16 issues and wrote fix pull request #28547. Niket Garg built the execution harness, the differential oracles, the first triage and the pinned development builds, and found and filed #27904. Janhvi Mittal built the static hint extraction, the coverage tooling, the schema and data generator and the seed extraction, and ran Round 2 of Method C. All authors reviewed the paper.

Use of Generative AI

LLMs are part of the method: Claude (Anthropic) and Gemini 2.5 Pro (Google) wrote the test generators of Methods C and D, as described in Section 3. In addition, substantial parts of the code in `src/harness`, the issue drafts and the text of this paper were written with the help of Claude under the direction of the authors. The authors checked every reproducer on the stated builds before filing, read every reported result, and take responsibility for the content.

References

Jinsheng Ba and Manuel Rigger. 2023. Testing Database Engines via Query Plan Guidance. In *Proceedings of the 45th International Conference on Software Engineering (ICSE)*.

Jinsheng Ba and Manuel Rigger. 2024. CERT: Finding Performance Issues in Database Systems Through the Lens of Cardinality Estimation. In *Proceedings of the 46th International Conference on Software Engineering (ICSE)*.

Marcel Böhme, Van-Thuan Pham, Manh-Dung Nguyen, and Abhik Roychoudhury. 2017. Directed Greybox Fuzzing. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*.

Jueon Eom, Seyeon Jeong, and Taekyoung Kwon. 2024. Fuzzing JavaScript Interpreters with Coverage-Guided Reinforcement Learning for LLM-Based Mutation. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*.

Mark Raasveldt and Hannes Mühleisen. 2019. DuckDB: an Embeddable Analytical Database. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)*.

Manuel Rigger and Zhendong Su. 2020a. Detecting Optimization Bugs in Database Engines via Non-Optimizing Reference Engine Construction. In *Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*.

Manuel Rigger and Zhendong Su. 2020b. Finding Bugs in Database Systems via Query Partitioning. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020).

Manuel Rigger and Zhendong Su. 2020c. Testing Database Engines via Pivoted Query Synthesis. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

Andreas Seltenreich. 2016. SQLsmith: a random SQL query generator. https://github.com/anse1/sqlsmith. Accessed: 2026-11-13.

Donald R. Slutz. 1998. Massive Stochastic Testing of SQL. In *Proceedings of the 24th International Conference on Very Large Data Bases (VLDB)*.

Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. 2024. Fuzz4All: Universal Fuzzing with Large Language Models. In *Proceedings of the 46th International Conference on Software Engineering (ICSE)*.

Chenyuan Yang, Yinlin Deng, Runyu Lu, Jiayi Yao, Jiawei Liu, Reyhaneh Jabbarvand, and Lingming Zhang. 2024. WhiteFox: White-Box Compiler Fuzzing Empowered by Large Language Models. *Proceedings of the ACM on Programming Languages* 8, OOPSLA2 (2024).

Rui Zhong, Yongheng Chen, Hong Hu, Hangfan Zhang, Wenke Lee, and Dinghao Wu. 2020. SQUIRREL: Testing Database Management Systems with Language Validity and Coverage Feedback. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS)*.